Quante volte la parola “stella” o “stelle” (case insensitive) compare nella Divina Commedia?

Divina Commedia in formato testo:
https://www.gutenberg.org/ebooks/1012.txt.utf-8

Votate su sli.do:
https://bit.ly/FAS-lez01es01

```
ubuntu@10b3cd3e33b2:/tmp/wordcount$
```

https://asciinema.org/a/5NdKWFVl7VfLGwzP

Fondamenti di Amministrazione di Sistema

Cristian Consonni

Università degli Studi di Trento – a. a. 2025/2026

Bash and CLI Tools

- Why Do We Care About This? It's $YEAR!
- *nix Filesystem Hierarchy
- Bash 101
- Bash Scripting
- CLI tools

Motivazione

- Perché imparare ad usare la linea di comando? È il $YEAR!
- Cosa vuol dire amministrare sistemi?
- E ChatGPT?

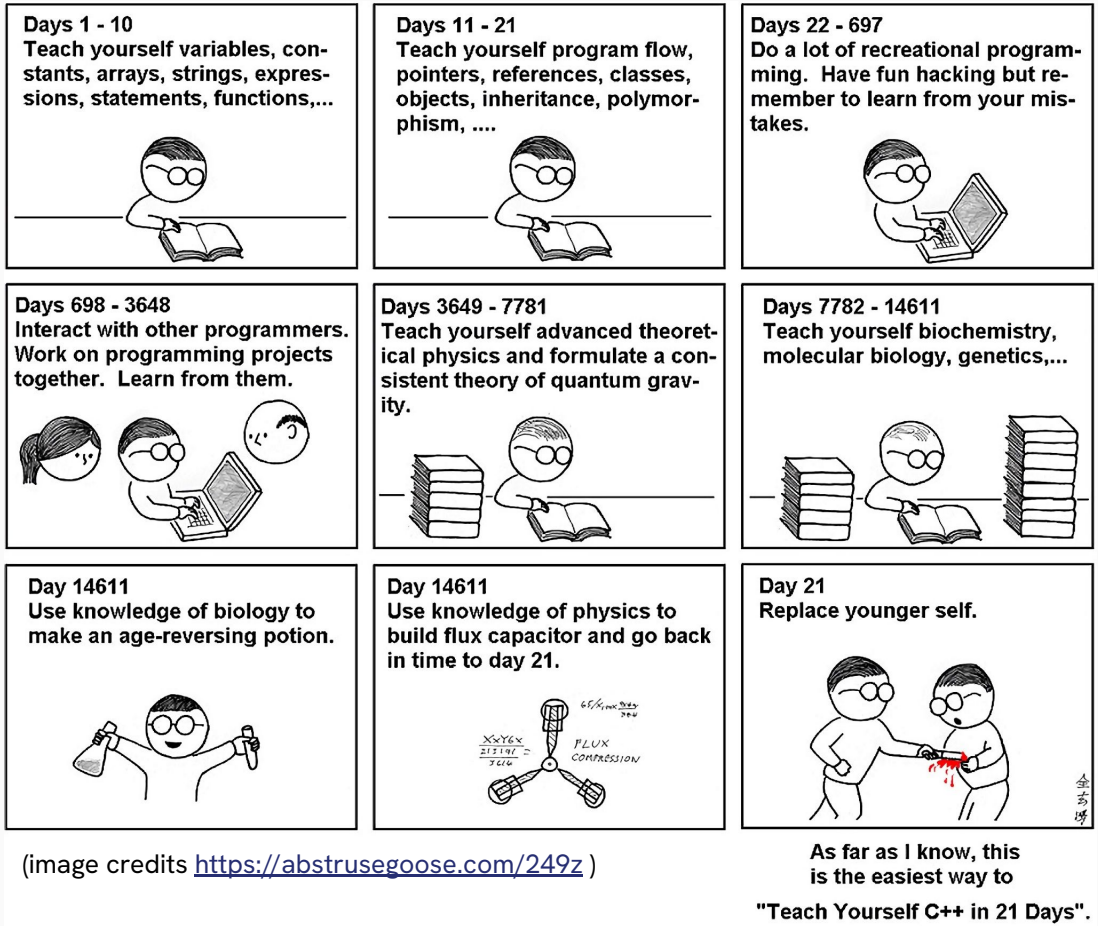
Perché imparare ad usare la linea di comando? È il $YEAR!

Una shell è un'interfaccia testuale universale che permette di eseguire programmi, fornire input e ispezionare l'output.

- Questi tool sono **molto** veloci:
  «Command-line Tools can be 235x Faster than your Hadoop Cluster» (link)
- I comandi possono essere **combinati in modo creativo**.
- Sono standard (POSIX) e disponibili quasi ovunque:
  - Eccezione #1: sh
  - Eccezione #2: Windows servers

Perché (II)

(si applica alla programmazione e anche all’amministrazione di sistema)



(image credits https://abstrusegoose.com/249z)

Cosa vuol dire amministrare sistemi?

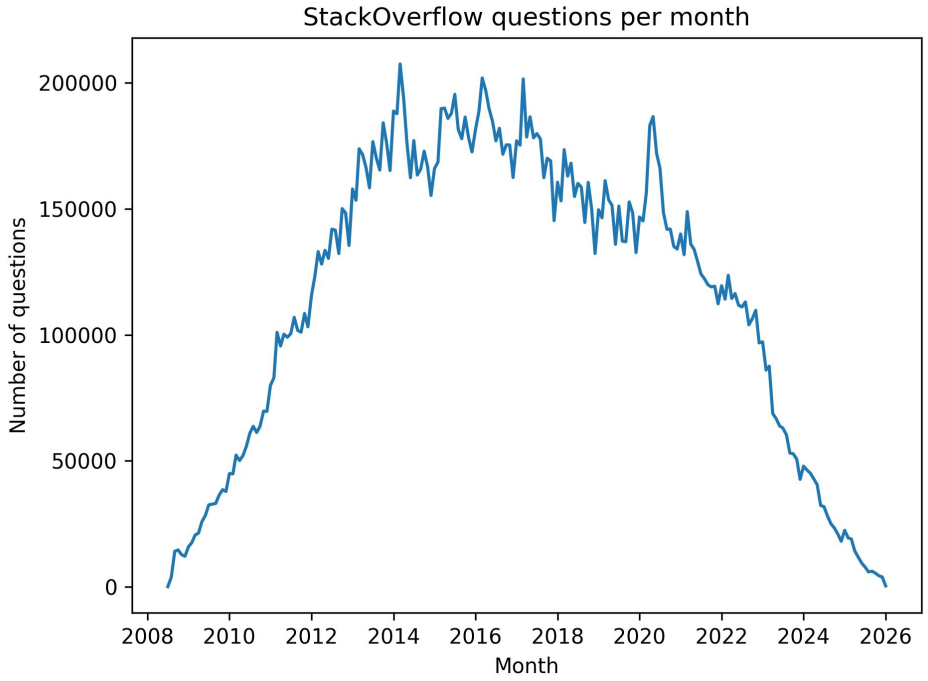
Tradizionalmente, amministrare un sistema significa gestire una o più macchine:

```
admin: /ad-min'/ n. Short for `administrator'; very commonly used in
speech on-line to refer to the systems person in charge on a computer.
Common constructions on this include `sysadmin' and `site admin'
(emphasizing the administrator's role as a site contact for email and
news) or `newsadmin' (focusing on the latter). Compare {postmaster},
{sysop}, {system mangler}.
```

-- The Jargon File, version 4.4.8, Ottobre 2004

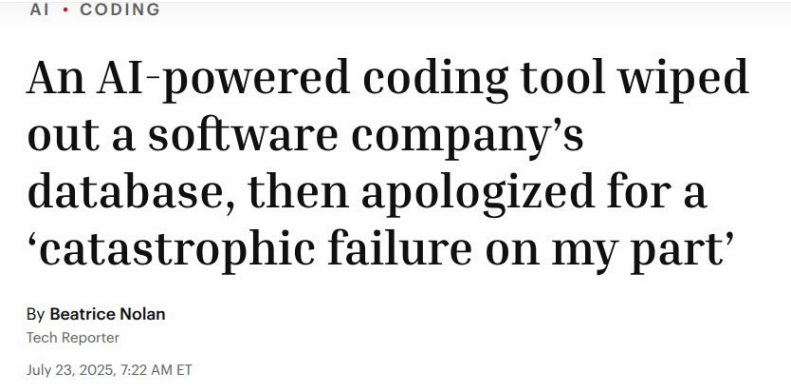
Ora significa anche amministrare sistemi distribuiti e in cloud

E ChatGPT? (I)



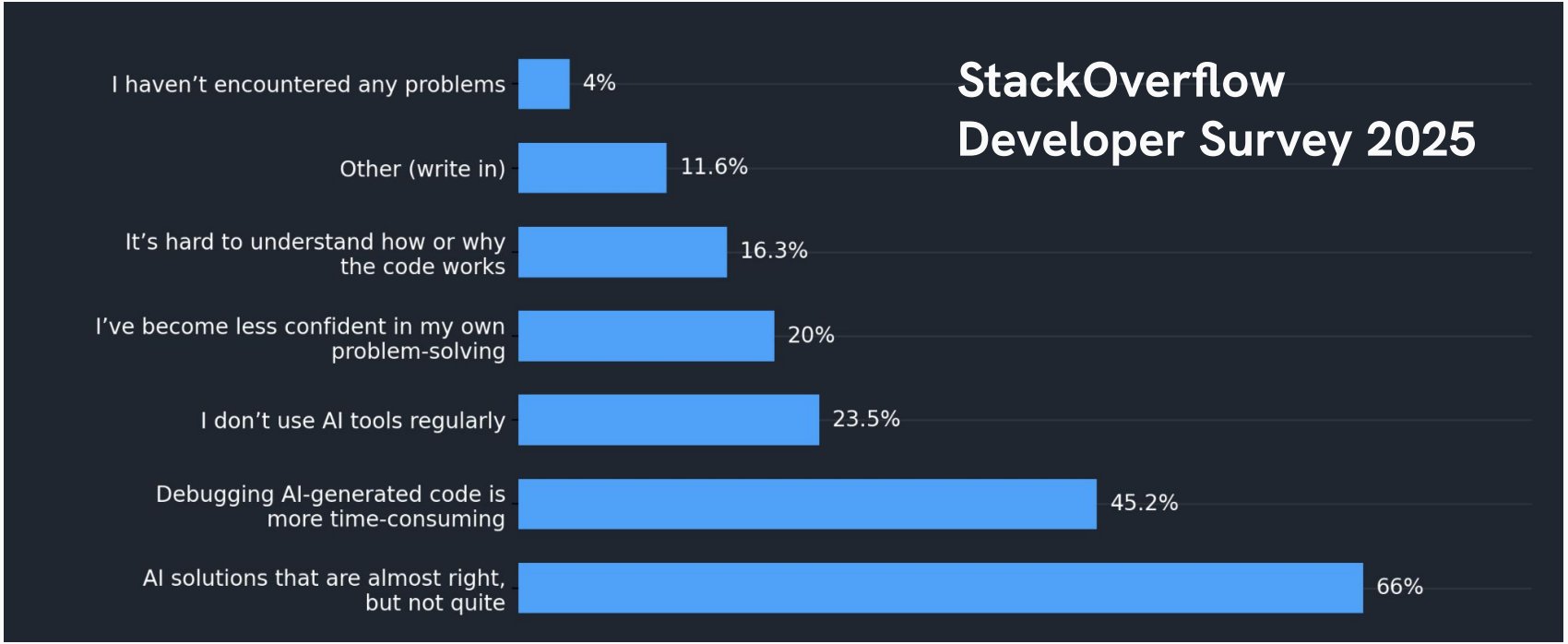
https://data.stackexchange.com/stackoverflow/query/1926661

https://www.reddit.com/r/google_antigravity/comments/1p82or6/google_antigravity_just_deleted_the_contents_of/



https://fortune.com/2025/07/23/ai-coding-tool-replit-wiped-database-called-it-a-catastrophic-failure/

E ChatGPT? (II)



https://survey.stackoverflow.co/2025/ai#developer-tools-ai-frustration

Policy sull’uso dell’AI

1. Potete usare i tool/modelli che preferite
2. Indicate esplicitamente quali sono i modelli che avete usato
3. L’esame consisterà in una presentazione dei vostri lavori, quindi dovete capire cosa consegnerete

Contenuti del corso

Parte 1
(W1-W8, 9 lezioni + 6 lab)

Sistemi single-machine

Progetto 1

Parte 2
(W9-W13, 6 lezioni + 1 lab)

Sistemi multi-machine & cloud

Progetto 2

Computer icon by Hilmy Abiyyu A from Flaticon

whoami

- Scientific Project Officer
  @ European Centre for Algorithmic Transparency (ECAT) (2023-)
- Data Scientist @ EURECAT, Barcelona (2019-2023)
- Dottorato @ DISI con il prof. Montresor (2019)
- Esercitatore Algoritmi e Strutture Dati (2015-)
- Wikipediano (2007-), membro del direttivo di Wikimedia Italia (2010-2017)
- Linux user (2005-)

Contatti

- e-mail: `cristian.consonni@unitn.it`
  - Ricevimento su appuntamento: lunedì 10:00-12:00
- sito web: https://cricca.disi.unitn.it/fas/

Caveat emptor

- È il primo anno che tengo questo corso:
  - aspettatevi varie correzioni di rotta
  - i feedback sono graditi
- Copriremo moltissimi argomenti, in generale, ciascuno verrà trattato nell'arco di una lezione e di un'esercitazione. Ogni lezione sarà una panoramica pensata per aiutarvi ad approcciare l'argomento che potrete approfondire autonomamente.
- Il corso introdurrà diversi software e tool. Quando si parla di software specifici, esiste sempre il rischio che i contenuti invecchino rapidamente. I temi che copriremo saranno un modo per esplorare paradigmi e concetti la cui validità non è intaccata dal tempo.

Modalità d’esame

- Due progetti:
  - Primo progetto: a metà Aprile, 1 settimana, individuale
  - Secondo progetto: a inizio giugno, 1 settimana, di gruppo (max 3 persone)

- Esame orale:
  - Presentazione dei progetti

Bash 101 (I)

- Il prompt `cristian@fas:~$` mostra utente, macchina, directory corrente e stato dell'utente
- comando `man` oppure opzione `--help` per documentazione dei comandi
- Percorsi assoluti, relativi e speciali
  - . → directory corrente
  - .. → directory padre
  - ~ → home dell'utente

```
cristian@fas:~$ man
What manual page do you want?
For example, try 'man man'.
cristian@fas:~$ date
Tue Jan 27 20:06:40 UTC 2026
cristian@fas:~$ date --help
Usage: date [OPTION]... [+FORMAT]
   or:  date [-u|--utc|--universal]
[MMDDhhmm[[CC]YY][.ss]]
Display date and time in the given FORMAT.
With -s, or with [MMDDhhmm[[CC]YY][.ss]], set the
date and time.

[...]
```

Bash 101 (II)

- Il prompt fornisce alcune informazioni (`$PS1`):
  - `user@host:~/dir` **`$`** (regular user)
  - `root@host:~/dir` # (privileged ser)
- Le variabili iniziano con `$: echo $SHELL; echo $LANG; …`
- Le virgolette doppie (“`pippo`”) e singole (‘`pippo`’) si comportano in maniera diversa: le variabili nelle stringhe con virgolette doppie sono risolte, le stringhe nelle virgolette singole sono *literal*.

Comandi fondamentali

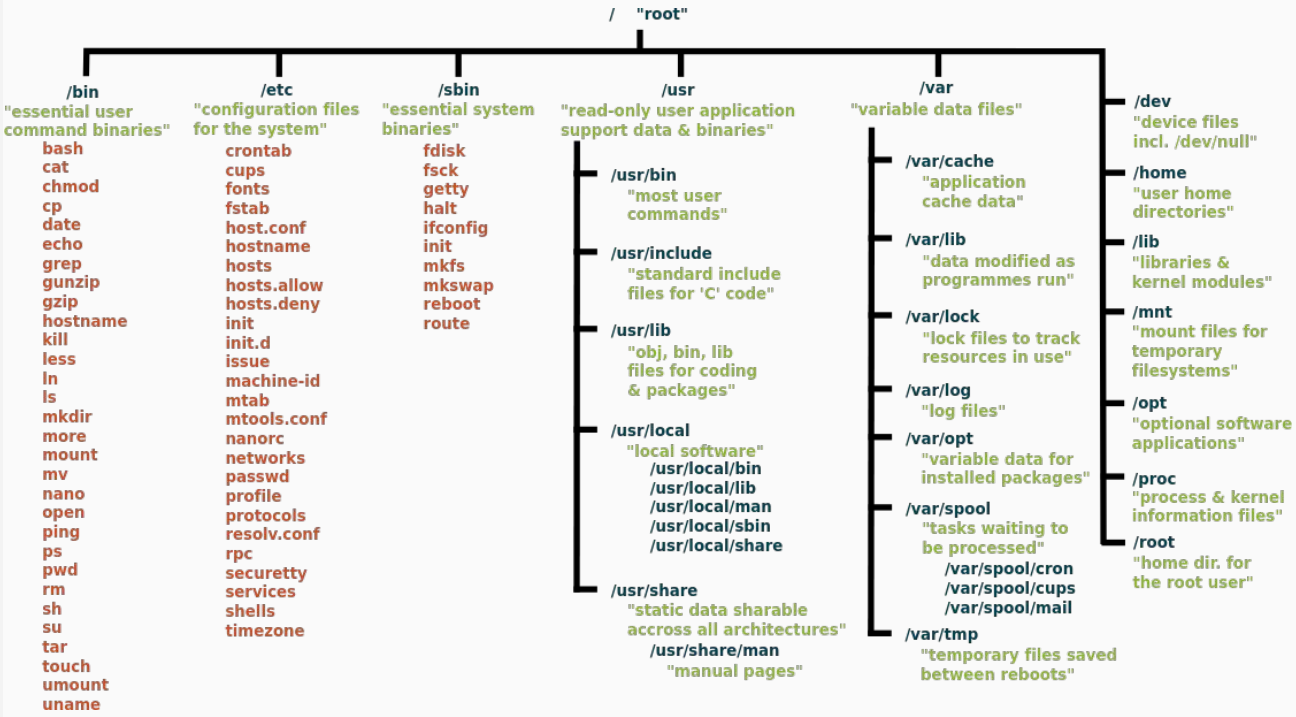
Comando	Origine nome	Funzione
`whoami`	who am I	quale utente è loggato
`pwd`	print working directory	dove sei
`echo <input>`	echo	stampa l'input a schermo
`which <command>`	which	percorso di un comando
`cat <file>`	concatenate	Stampa il contenuto di un file
`sort <file>`	sort	Ordina le righe
`uniq <file>`	unique	Elimina i duplicati consecutivi
`head <file>`	head	Visualizza le prime linee
`tail <file>`	tail	Visualizza le ultime linee
`grep <pattern> <file>`	`g/re/p`	Cerca righe che corrispondono a un pattern

Bash 101 (III)

- Valori di ritorno: `0` indica successo:
  - **[ATTENZIONE]** `0` è `true`, qualsiasi altro valore è `false`
- globbing/wildcards:
  - `*`, zero o più caratteri
    per es. `*.csv` trova `foo.csv`, `test.csv`, `bar.csv` ma non `bar.tsv`
  - `?`, un singolo carattere
    per es. `test??.csv` trova `test01.csv`, `testxy.csv`, ma non `test999.csv`
  - **[ATTENZIONE] il** globbing non considera i files nascosti
- `export VAR='foo'`

*nix Filesystem Hierarchy

Filesystem Hierarchy Standard (wiki)



Ppgardne, CC BY-SA 4.0, via Wikimedia Commons

Variabile PATH e ricerca programmi

- La variable `PATH` contiene directory separate da due punti in cui la shell cerca i programmi.
- `echo $PATH` per vedere l'ordine dei percorsi, e `which <command>` per trovare il percorso dell'eseguibile.
- Percorsi assoluti evitano la ricerca nel `PATH`; percorsi relativi dipendono dalla directory corrente.

Directory	Descrizione
`/usr/local/sbin`	Amministrazione di sistema locale
`/usr/local/bin`	Programmi installati dall'utente
`/usr/sbin`	Amministrazione di sistema
`/usr/bin`	Programmi principali del sistema
`/sbin`	Programmi di sistema essenziali
`/bin`	Comandi base

Comandi: find

`find`
Trova file in modo ricorsivo con condizioni.
`find ~/Downloads -type f -name "*.pdf" -mtime +30`
cerca file PDF più vecchi di 30 giorni.

```
$ LC_ALL=C journalctl -u ssh \
  | grep ‘Disconnected from authenticating user’ \
  | sed -E 's/.*user (.*) [^ ]+ port.*/$1/' \
  | sort \
  | uniq -c \
  | sort -nk1,1 \
  | tail -n10
```

Comandi: awk

- `awk`, pattern scanning and processing language
  - **[ATTENZIONE]** can't distinguish between content and delimiter
- `csvkit` (link), work with CSV files

```
$ head sample-homes.tsv
Sell   Rooms   Beds   Baths
142    10      5      3
175    8       4      1
129    6       3      1
138    7       3      1

$ awk '{print $3}' sample-homes.tsv
Beds
5
4
3
3

$ csvcut -t -c 3 sample-homes.tsv
Beds
5
4
3
3
```

Comandi: sed

- **sed**, stream editor per filtrare e trasformare il test. Modificare file in modo non interattivo.

```
sed -i \
  's/pattern/sost/g' \
   <file>
```

sostituisce tutte le occorrenze di `pattern` con `sost`.

```
$ grep -n -i 'grasshopper' grasshopper.txt
1:The Grasshopper
3:A grasshopper spent the summer hopping about in
the sun and singing to his
6:"Why are you working on such a lovely day?"
said the grasshopper.
10: The grasshopper carried on hopping and
singing. When winter came the ground
11: was covered with snow. The grasshopper had no
food and was hungry. So he went

$ sed -e 's/grasshopper/grillo/g' grasshopper.txt
| grep -i saltamontes
A grillo spent the summer hopping about in the
sun and singing to his
"Why are you working on such a lovely day?" said
the grillo.
 The grillo carried on hopping and singing. When
winter came the ground
 was covered with snow. The grillo had no food
and was hungry. So he went
```

CLI Tools (III)

- `jq`, command-line JSON processor

```
$ jq . example.json
{
  "name": "John",
  "age": 30,
  "car": null
}
```

Pipes e redirezione

Pipe (composizione)
Con l'operatore `|` l'output di un programma diventa l'input del successivo, permettendo di combinare vari comandi in successione.

Redirezioni
File descriptors, 1 è standard output (`stdout`), 2 è standard error (`stderr`):

- `2>/dev/null` mutare errors:
- `> output.txt` redirect stdout a un file, `>>` appende al file
- `< input.txt` per fornire input

Il comando `tee` duplica l'output verso un file e il terminale.

Composizione tramite pipe

Bash scripting (I)

- Il simbolo `#!` si chiama “shebang” e serve per specificare l’interprete `#!/bin/bash`
- Opzioni della shell impostate con `set` «Use Bash Strict Mode (Unless You Love Debugging)» (link)
- `IFS` (internal field separator) indica i caratteri interpretati come separatori.

```
#!/bin/bash

set -euo pipefail
IFS=$'\n\t'
```

Bash scripting (II)

Attiva modalità stretta con set -euo pipefail per intercettare errori

Utilizza variabili sicure e controlli condizionali per gestire gli errori

Esegui attività in background con & per parallelizzare le operazioni

Documenta e commenta per rendere gli script mantenibili

```bash
#!/bin/bash
set -euo pipefail

# Avvia stress in background
stress --cpu 8 &
STRESS_PID=$!

# Esegui test fino a fallimento
RUN=1
while cargo test my_test > "$LOG" 2>&1; do
  echo "Run $RUN riuscito"
  ((RUN++))
done

# Pulizia e report finale
kill $STRESS_PID
```

Bash scripting (III)

- `if-then-else`
- `for` loop
- `while` loop

Controllo di flusso
Bash supporta if/else, while e for. La sintassi $(cmd) consente la sostituzione di comandi per costruire loop dinamici.

```
if $var; then
  echo “$var is true”
fi

for afile in ./*.txt; do
  head “$afile”
done

while read -r aline; do
  ...
done
```

Bash scripting: exit traps

- Exit traps (link), a function that gets called when the script exists
  - Remove temporary files
  - Stop virtual machine/containers

```bash
#!/bin/bash

scratch=$(mktemp -d -t tmp.XXXXXXXXXX)
function finish {
  rm -rf "$scratch"
}
trap finish EXIT
```

Conclusioni

- Essere informatici può voler dire molte cose (algoritmi, SO, sistemi embedded e sensori, AI e machine learning, data science, ecc.)
- Richiede una varietà di competenze e strumenti per svolgere il lavoro, ci sono molti strumenti che vi possono aiutare giorno per giorno.
- (Prima parte del corso) Single-machine → (Seconda parte del corso) Multi-machine